

MARS：面向飞书群聊与 Agent 的外挂式 Memory Engine 关键设计整理

1. 项目定位

MARS (Memory Agent for Reconciliation and Summoning) 是一个独立于具体 Agent 的外挂式 Memory Engine，用于解决企业协作场景中的“长期记忆治理”问题。

它不是：

- 普通聊天记录搜索工具
- 简单 RAG 系统
- OpenClaw 内部的记忆文件夹
- 单纯 Prompt Engineering

它是：

- Agent 外部记忆基础设施
- 企业级状态化记忆治理系统
- 多 Agent 可复用的统一 Memory 插口
- 面向飞书协作场景的长期记忆管理层

核心设计思想：

Feishu = 数据源与交互界面
OpenClaw / Agent = 调用方
MARS = 独立 Memory Engine

其目标并不是“存聊天记录”，而是：

把飞书群聊、文档、会议、任务中的碎片化历史，
转化为可检索、可追溯、可更新、可冲突协调、可主动唤醒的团队记忆。

2. 项目解决的问题

2.1 企业协作中的“失忆”问题

企业团队在飞书中每天会产生大量内容：

- 项目群聊
- 技术讨论
- 接口变更
- 需求修改
- 会议纪要

- 截止日期
- 风险提醒
- 周报与复盘

随着时间增长，会出现：

- 历史决策遗忘
- 重复讨论
- 旧版本信息复活
- 新人无法理解上下文
- 群聊搜索成本高
- 责任人和截止日期混乱

普通 Agent 即使推理能力很强，如果每次对话都从零开始，本质上仍然是“失忆”的。

MARS 的核心目标就是：

把历史真正转化为当前决策可用的信息。

2.2 为什么普通 RAG 不够

传统 RAG：

文本切块 → 向量检索 → 拼 Prompt → LLM 回答

存在的问题：

- 没有版本治理
- 没有冲突处理
- 没有记忆生命周期
- 没有主动提醒
- 没有来源审计
- 没有时间状态管理
- 不知道“哪个是当前有效结论”

因此：

普通 RAG 解决的是“能否查到”；
MARS 解决的是“什么值得记、什么时候更新、哪些应该失效”。

3. 系统总体架构

3.1 核心闭环

MARS 的完整记忆闭环：



整个系统强调：

历史 → 结构化 → 状态化 → 可治理 → 可影响当前决策

3.2 八层架构

MARS 被划分为八个核心层：

层	作用
Interface Layer	接入外部系统
Raw Ledger Layer	保存原始事件
Memory Extraction Layer	提取长期记忆
View Layer	构建不同记忆视图
Retrieval Layer	检索相关记忆
Policy Layer	控制读写更新策略

层	作用
Reconciliation Layer	处理冲突与版本
Summoning Layer	主动推送历史记忆
Benchmark Layer	评测系统有效性

4. MARS 的核心理念

4.1 Memory 不是存储，而是“历史到决策”的通道

MARS 的关键观点：

Memory 的价值不在于存了多少历史，而在于历史能否正确影响当前决策。

因此：

- Memory 不等于数据库
- Memory 不等于聊天记录
- Memory 不等于 Prompt 拼接

真正重要的是：

- 状态机
- 生命周期
- 来源追踪
- 冲突治理
- 主动唤醒
- 检索策略
- 语义版本控制

4.2 Raw Ledger + Derived Views + Policy

MARS 的核心三件套：

(1) Raw Ledger

原始账本。

保存：

- 谁说了什么
- 在哪里说的
- 什么时间说的

- 后来有没有变化
- 来源证据是什么

特点:

- append-only
- 可审计
- 可回放
- 可追溯
- 双时态 (valid_time + transaction_time)

它是系统唯一的 source of truth。

(2) Derived Views

从原始账本衍生出的各种记忆视图:

- 决策卡片
- 事实卡片
- 流程卡片
- 风险卡片
- Timeline
- 向量索引
- 飞书多维表格
- Markdown 镜像

本质上:

Raw Ledger 是原始事实;
Derived Views 是面向不同用途的可读形态。

(3) Policy

Policy 控制:

- 什么时候读 Memory
- 什么时候写入
- 写成什么类型
- 什么时候更新
- 什么时候遗忘
- 什么时候主动推送

Policy 是整个系统的“治理层”。

5. LLM 与规则系统的分工

MARS 不是全靠 LLM。

它采用：

LLM 做语义；
代码做治理；
数据库做状态；
Policy 做调度。

5.1 LLM 负责

LLM 主要承担：

- 候选记忆抽取
- Memory Object 生成
- 新旧记忆关系判断
- 推送卡片生成
- 新人 onboarding 包生成
- Benchmark GPT-as-judge

即：

LLM 负责理解“语义”。

5.2 规则与代码负责

代码负责：

- RawEvent 入库
- 状态机流转
- 版本控制
- active/superseded 更新
- 检索逻辑
- 频控
- 来源追踪
- benchmark 统计

即：

代码负责“确定性治理”。

6. 多 Agent 统一接口设计

6.1 Agent-Agnostic 设计

MARS 最重要的设计之一：

Memory Engine 必须与 Agent 解耦。

因此它不绑定：

- OpenClaw
- Claude
- Codex
- CLI
- 飞书 Bot

所有系统都通过统一 Adapter 接入。

6.2 Adapter 机制

外部输入统一转换为 RawEvent：

```
sample JSON → sample_adapter → RawEvent
OpenClaw context → openclaw_adapter → RawEvent
Feishu event → feishu_event_adapter → RawEvent
CLI input → cli_adapter → RawEvent
```

因此理论上：

任何 Agent 只要遵守 RawEvent Schema，
都可以共享同一套长期记忆系统。

这是整个项目最重要的扩展性来源。

7. 群聊 Memory 的核心难点

MARS 设计中特别强调：

飞书群聊不是普通文档。

它具有：

- 多人交互
- 讨论发散
- 多轮修改

- 上下文跳跃
- 情绪与闲聊混杂
- 时间跨度长
- Thread 与 Reply 结构

因此：

不能按单条消息直接提取 Memory。

7.1 Window Builder

系统先构建“讨论窗口”：

输入可能是：

- 最近 N 条消息
- 一个 Thread
- 某时间段聊天
- 一个会议纪要

切分方式：

- 时间窗口
- Thread 聚合
- Topic 聚类
- 用户命令触发

因为真正的“决策”往往来自：

发散讨论 → 反对意见 → 收束确认 → 后续修正

而不是单条消息。

7.2 Candidate Extractor

LLM 会从讨论窗口中提取：

- 决策
- 风险
- 事实
- 流程
- 负责人
- 截止日期

同时必须：

- 给出证据事件
- 给出置信度
- 判断是否需要人工确认

不提取：

- 闲聊
- 情绪表达
- 未确认个人建议

8. Memory Object 设计

MARS 的长期记忆不是普通文本，而是结构化对象。

核心字段包括：

字段	含义
memory_type	决策/事实/风险等
scope	project/team/user
topic	主题
rationale	为什么这样决定
objections	反对意见
confidence	置信度
importance	重要性
status	当前状态
source_event_ids	来源证据
supersedes	覆盖关系

重点不是“记住一句话”，而是：

记住：

- 最终结论
- 为什么
- 谁反对过
- 是否仍有效
- 来源在哪

9. Git-like Memory Governance

MARS 最大创新之一：

把 Git 的版本治理思想引入语义记忆系统。

对应关系：

Git	MARS
commit	raw_event / policy_action
diff	semantic memory diff
merge	memory merge
conflict	conflicted memory
revert	correction / supersede
blame	provenance trace
log	memory timeline
branch	project / phase

关键点：

MARS 管理的不是文本，
而是“语义化记忆对象”。

因此系统能够：

- 识别旧结论被新结论覆盖
- 管理冲突状态
- 保存版本链
- 记录来源与演化过程

10. Reconciliation：冲突与覆盖

这是 MARS 的核心层之一。

系统会判断新旧记忆关系：

关系	含义
duplicate	重复
support	支持
update	补充
conflict	冲突
supersede	明确覆盖

关系	含义
unrelated	无关

例如：

旧：一期使用 `Streamlit`
新：不对，一期改成 `Vue + FastAPI`

系统会：

`old_memory.status = superseded`
`new_memory.status = active`
建立 `memory_edge`
写入 `policy_action`

从而保证：

系统始终知道“当前有效版本”是什么。

11. Memory 生命周期

MARS 的 Memory 是状态化的。

状态包括：

状态	含义
pending	待确认
active	当前有效
superseded	已被覆盖
conflicted	存在冲突
expired	已过期
archived	已归档
rejected	已拒绝

因此：

Memory 不是“写进去永远有效”。

系统会不断：

- 更新
- 覆盖

- 降权
- 归档
- 复习提醒

12. Retrieval: 检索设计

MARS 不采用单纯向量检索。

而是：

Hybrid Retrieval

包含：

- 项目过滤
- 时间过滤
- 状态过滤
- 关键词检索
- 向量检索
- rerank
- evidence assembling

最终返回：

Evidence Pack

其中包含：

- 当前有效记忆
- 来源消息
- 理由
- 反对意见
- 时间信息
- 置信度

本质上：

MARS 返回的不只是答案；
而是“带证据的答案”。

13. Summoning: 主动唤醒

MARS 不只是被动搜索。

它会在合适的时候主动提醒。

例如群聊中出现：

- “之前是不是已经决定？”
- “为什么不用 Vue？”
- “负责人是谁？”
- “要不要改方案？”

系统会主动推送：

【历史决策提醒】
此前团队已确认一期采用 Streamlit...

并附带：

- 理由
- 来源
- 当前状态
- 是否需要重新打开决策

这是 MARS 与普通知识库最大的区别之一。

14. Forgetting: 遗忘机制

MARS 不做物理删除。

而是：

- 降权
- archived
- expired
- superseded
- review reminder

MemoryScore：

MemoryScore =
Importance × Confidence × Freshness × AccessFactor × StatusFactor

这样可以避免：

旧事实不断污染当前决策。

15. Benchmark 评测设计

MARS 强调：

Memory 系统必须证明自己真的有效。

因此设计了 Benchmark：

15.1 抗干扰测试

过程：

Day1 写入关键记忆
Day2-7 注入大量噪声聊天
Day7 提问

指标：

- Recall@k
- Source Accuracy
- Noise Robustness

15.2 冲突更新测试

测试系统是否能正确覆盖旧结论。

指标：

- Conflict Detection Accuracy
- Supersede Accuracy
- Temporal Accuracy

15.3 效能测试

比较：

人工翻群聊
VS
MARS 自动检索与推送

指标：

- 查找时间
- 操作步数
- 输入字符数

- 重复讨论减少率

16. 当前 MVP 实现状态

当前实现内容：

- FastAPI 骨架
- SQLite Schema
- RawEvent 模型
- MemoryObject 模型
- sample_loader
- 数据入库
- 基础项目结构

当前仍是 Placeholder 的部分：

- 真正的 LLM 提取
- 检索系统
- Reconciliation
- Benchmark 执行
- 飞书实时接入
- OpenClaw Runtime 集成

当前目标：

先完成本地可运行 MVP；
后续再接真实飞书与 Agent。

17. 整个项目最核心的价值

MARS 的真正意义并不是：

“让 Agent 能记住聊天记录”

而是：

让企业协作中的长期知识与历史决策，
变成可治理、可演化、可追溯、可主动参与未来决策的系统级 Memory。

其本质已经不只是“外挂记忆”。

而是在构建：

企业级 Agent Memory Infrastructure

也就是：

企业协作场景中的长期认知层。

18. 从架构角度理解整个系统

一句话总结：

MARS = 面向多 Agent 的、状态化的、可治理的、Git-like 的企业长期记忆系统。

其核心创新点：

- Memory Object 化
- Git-like 语义版本治理
- Raw Ledger + Derived Views
- Reconciliation 冲突协调
- Summoning 主动唤醒
- Agent-Agnostic 接口设计
- 面向飞书群聊的窗口化记忆抽取
- 长期生命周期与遗忘机制
- 带来源证据的检索系统
- Benchmark 驱动的可验证设计

19. 与其他memory系统的对比

可以从架构理念、功能定位、Agent 兼容性和治理机制几个维度来看 MARS 与 mem0 或其他成熟 memory 系统的区别。我给你整理得比较清楚：

1. 架构定位

系统	架构特点	特点总结
MARS	外挂式、Agent-agnostic; Raw Ledger + Derived Views + Policy + Reconciliation + Summoning	独立于任何具体 Agent 或 LLM；面向企业协作，特别是飞书群聊、文档和会议纪要；支持主动唤醒和冲突治理
mem0 / LLM internal memory	内嵌于 Agent 或 LLM；通常为 KV-store / vector store	Agent 依赖 Memory 内部状态；偏向短期上下文保存和即时检索；缺乏复杂治理策略

系统	架构特点	特点总结
其他成熟 memory 系统 (如 LangChain Memory, BabyAGI Memory)	模块化但通常紧耦合 LLM, 支持 RAG 或简单持久化	强调检索 + prompt 回填; 生命周期管理通常依赖外部策略; 不强调语义版本治理

关键区别：MARS 是企业级、外部化、可被多 Agent 调用；而 mem0 和大部分成熟 memory 系统更多是 Agent 内部状态或轻量持久化，缺乏跨 Agent 通用性和复杂治理。

2. 数据与记忆治理

维度	MARS	mem0 / 内部 memory	其他成熟 memory 系统
数据源	飞书群聊、文档、会议、用户操作	Agent 上下文、对话缓存	Agent 对话、数据库或 RAG 文本块
版本治理	Git-like: raw_event commit / semantic diff / conflict / supersede / provenance trace	通常无版本管理或仅存 timestamp	多为简单 overwrite 或 append-only; 少有冲突标记
冲突处理	Reconciliation 层: duplicate / support / update / conflict / supersede / unrelated	无显式冲突管理	有些系统提供简单更新或覆盖, 但缺少系统化规则
溯源	每条 MemoryObject 都关联原始 RawEvent, 可审计	内部 KV / vector store 有 trace, 但通常不面向多人协作	多为单 Agent trace, 缺少多用户来源审计

3. 生命周期和策略控制

功能	MARS	mem0 / 内部 memory	其他成熟 memory 系统
状态机	pending → active → superseded / expired / archived / conflicted / rejected	通常只有 active 或过期	少数实现状态机, 多为 overwrite 或 TTL
主动唤醒	Summoning Layer: 触发词、语义、时间、遗忘、冲突	大多数 memory 无主动推送机制	一般靠定时或调用触发, 不内置复杂策略
遗忘 / 复习	Forget Policy: 降权 / archived / 超期 / supersede / 复习提醒	TTL 或人工清理	基于 TTL 或外部触发, 缺少主动提醒

4. Agent 与多用户协作

特性	MARS	mem0 / 内部 memory	其他成熟 memory 系统
Agent 无关	✔ 可以被 OpenClaw / Feishu Bot / CLI 等多入口调用	✘ 绑定单 Agent	多数绑定特定 LLM / Agent
多用户支持	✔ 支持 tenant / project / team / user 维度	✘ 通常单用户或单 Agent	部分支持简单多用户标签
协作可视化	✔ 飞书多维表格 / Markdown 镜像 / Timeline	✘ 内部状态无可视化	部分 RAG 系统提供简单 Web UI

5. 总结对比

1. MARS 核心优势

- 企业级、多入口、跨 Agent
- Git-like 版本治理，冲突覆盖，溯源追踪
- Memory 生命周期管理和主动推送
- 多维视图（决策卡片、事实、流程、风险）
- 面向团队协作，尤其是飞书场景

2. mem0 / 内部 memory 优势

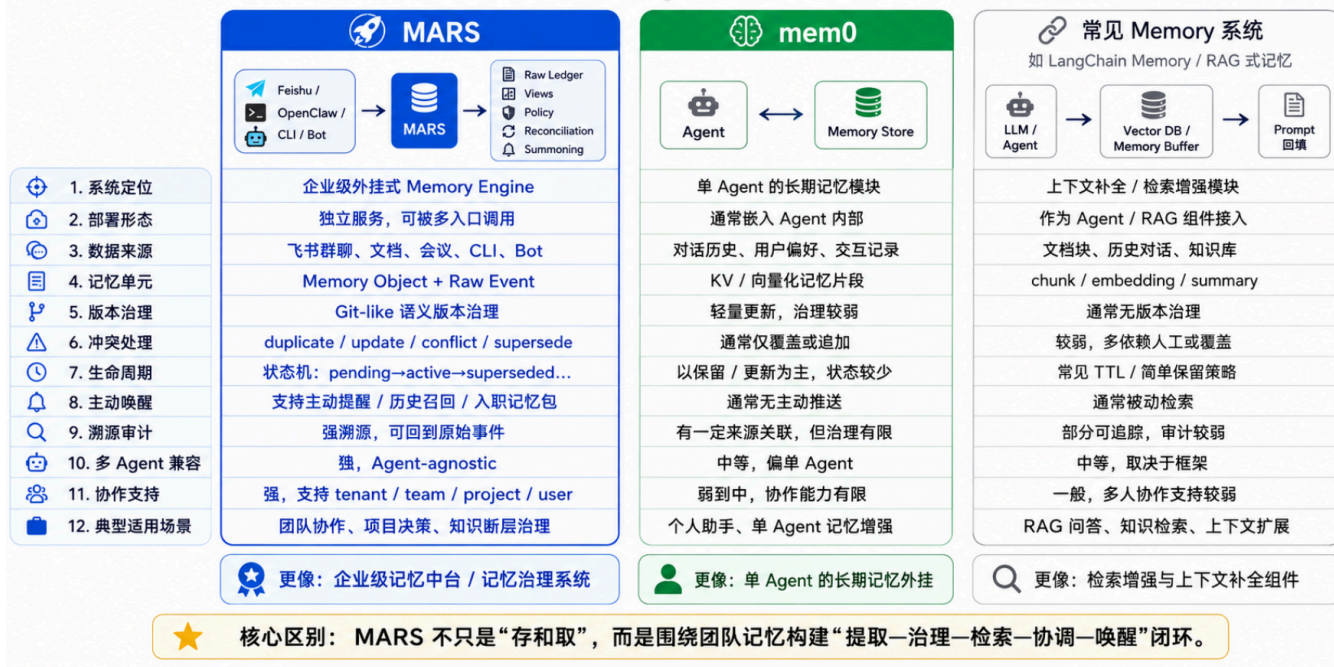
- 紧耦合 LLM，调用低延迟
- 轻量存储，简单 KV / vector
- 易集成到单 Agent 流程

3. 主要区别点

- 治理能力：MARS > mem0
- 多 Agent 通用性：MARS > mem0
- 主动性与生命周期：MARS > mem0
- 复杂多源协作：MARS 明显优势

MARS 与 mem0 / 常见 Memory 系统对比

面向飞书群聊与多 Agent 场景的外挂式记忆引擎



MARS: 企业级外挂式记忆引擎 (Memory Agent for Reconciliation and Summoning)

核心定位:

- 独立于具体 Agent 或 LLM 的记忆管理系统, 可被 OpenClaw、飞书 Bot、CLI 等多入口调用。
- 将飞书群聊、文档、会议纪要和用户操作中的碎片化历史信息, 转化为可追溯、可检索、可更新、可遗忘、可主动唤醒的团队记忆。
- 解决企业协作中的“信息散落、历史决策遗忘、冲突管理困难、团队知识断层”等问题。
- 与普通 RAG 系统或内部 memory 不同, MARS 不仅“存和取”, 更强调治理闭环: 提取 → 治理 → 检索 → 协调 → 唤醒。

核心价值:

1. **多 Agent 通用**: 支持不同 Agent 调用, 无需修改底层逻辑。
2. **语义版本治理**: Git-like 管理 Memory Object, 支持 duplicate / update / conflict / supersede。
3. **主动唤醒与复习**: 根据触发词、语义、时间或冲突, 主动推送历史决策或重要信息。
4. **多维视图**: 决策卡片、事实卡片、流程、风险、向量索引、Markdown 镜像、飞书多维表格。
5. **可溯源**: 每条记忆关联原始事件, 支持审计、回放和来源追踪。

MARS 核心架构与流程

1. **数据接入层 (Interface Layer)**
 - 接收飞书群聊、文档、CLI 或 OpenClaw 输入
 - 将外部事件统一转换为 Raw Event
2. **原始账本层 (Raw Ledger Layer)**
 - append-only 的事实来源, 支持双时态
 - 可回溯、可审计、可重放
3. **记忆提取层 (Memory Extraction Layer)**

- 通过讨论窗口抽取候选记忆
- 将候选记忆整理为结构化 Memory Object
- 判断新旧记忆关系 (duplicate / support / update / conflict / supersede)

4. 派生视图层 (Derived Views)

- 生成决策卡片、事实卡片、流程、风险、时间线
- 支持关键词索引、向量索引、Markdown 导出、飞书多维表格

5. 策略控制层 (Policy Layer)

- 决定何时读取、写入、更新、遗忘和主动推送
- 结合规则 + LLM 语义判断

6. 记忆协调层 (Reconciliation Layer)

- 管理冲突、补充、覆盖
- 维护版本链、溯源追踪

7. 主动唤醒层 (Summoning Layer)

- 根据触发词或语义主动提醒历史决策
- 支持新人入职记忆包和复习提醒

8. 评测层 (Benchmark Layer)

- 验证系统抗干扰能力、冲突更新正确率和效率提升